

Intro to High-Performance-Computing

Dr Jake Mac Uilliam

What is HPC?

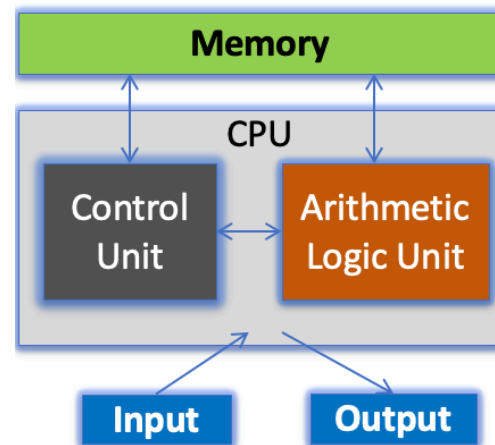
- High-Performance Computing (HPC) encompasses a collection of
 - powerful hardware systems and fast networks
 - high/low level libraries and languages
 - software tools
 - programming paradigms
- These combine to make previously unfeasible calculations possible.
- HPC is all about **SIZE** and **SPEED**.

Why HPC?

- HPC can benefit those who wish to carry out repetitive calculations and obtain results in a reasonable time.
- HPC supplements traditional scientific and engineering methods and can be used to replace experiments that are
 - too expensive
 - too slow
 - too dangerous
 - not possible

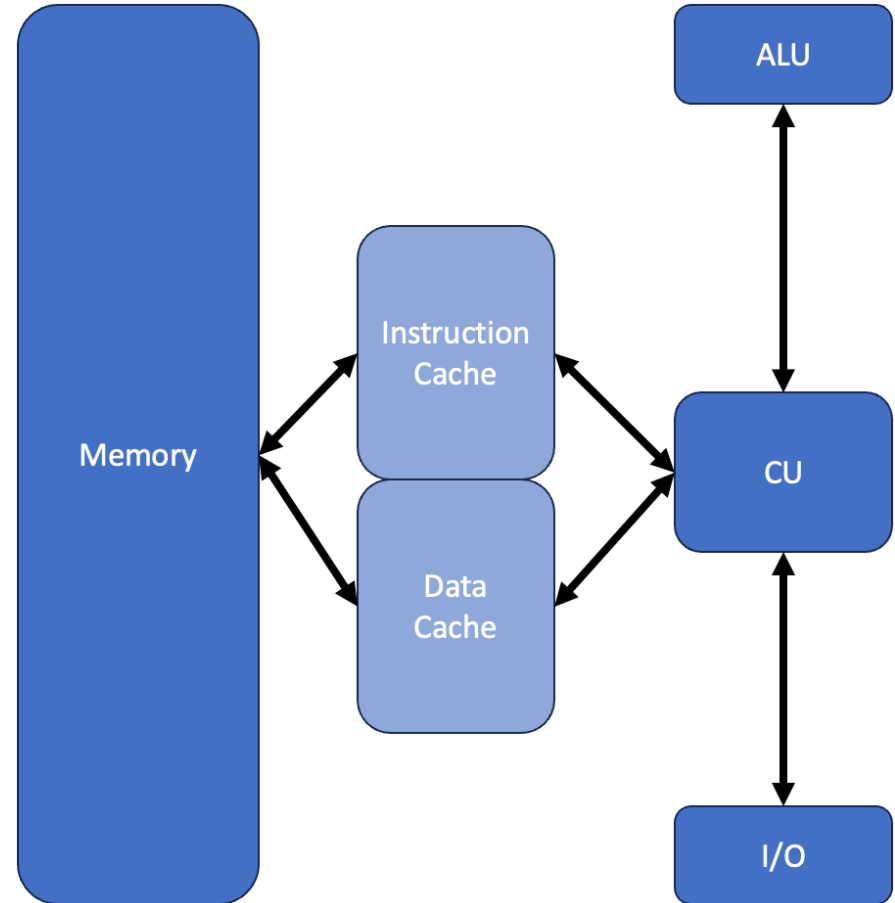
von Neumann Architecture

- Architecture of a computer proposed by Hungarian-American mathematician John von Neumann
- Part of the EDVAC, which followed ENIAC
- Four main components:
 - Memory
 - Control Unit (CU)
 - Arithmetic Logic Unit (ALU)
 - Input/Output

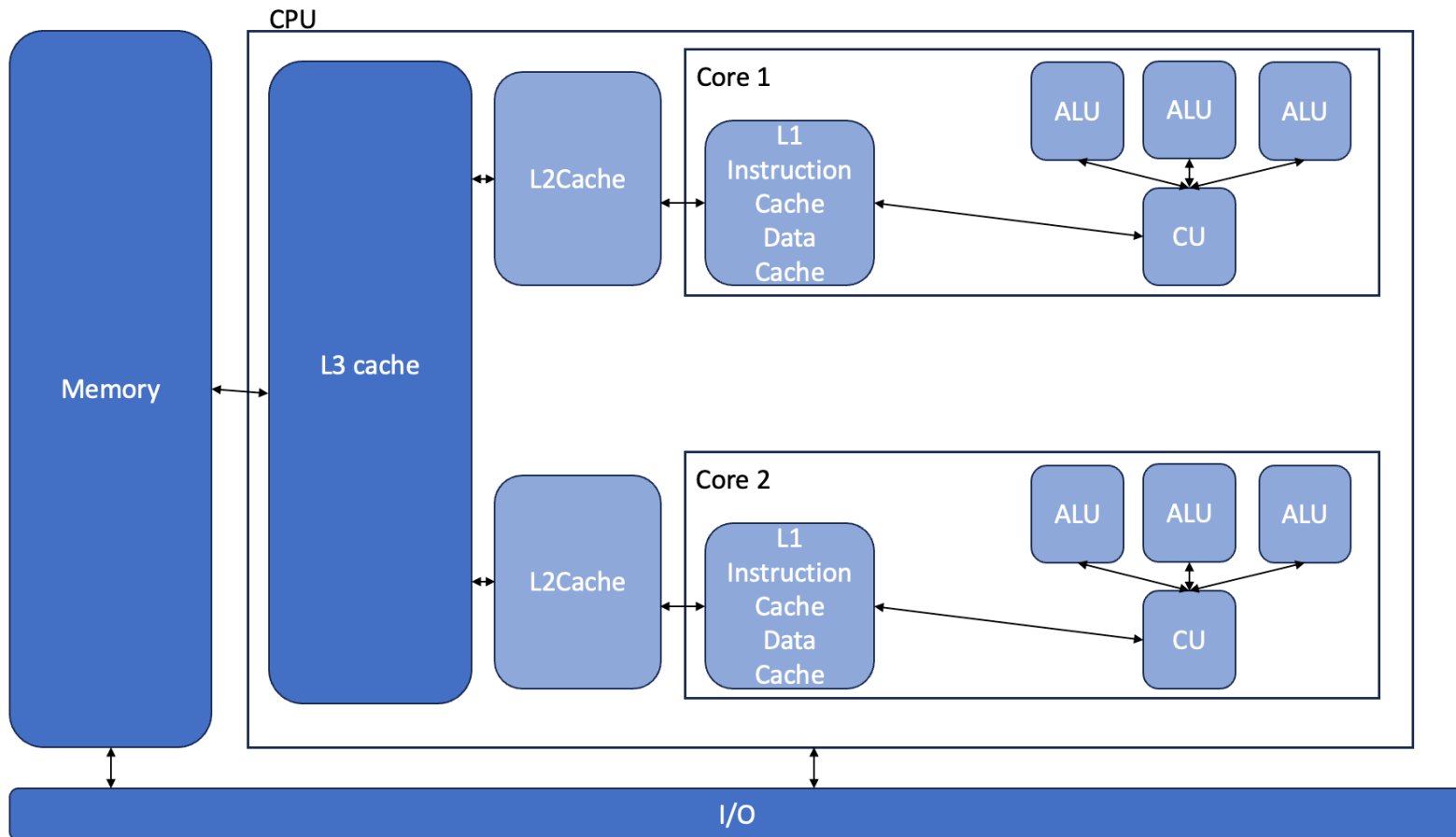


Modified Harvard Architecture

- Instruction read and data memory access can be simultaneous
- Different address spaces for program and data memories
- Caching system with short access latency near the CPU



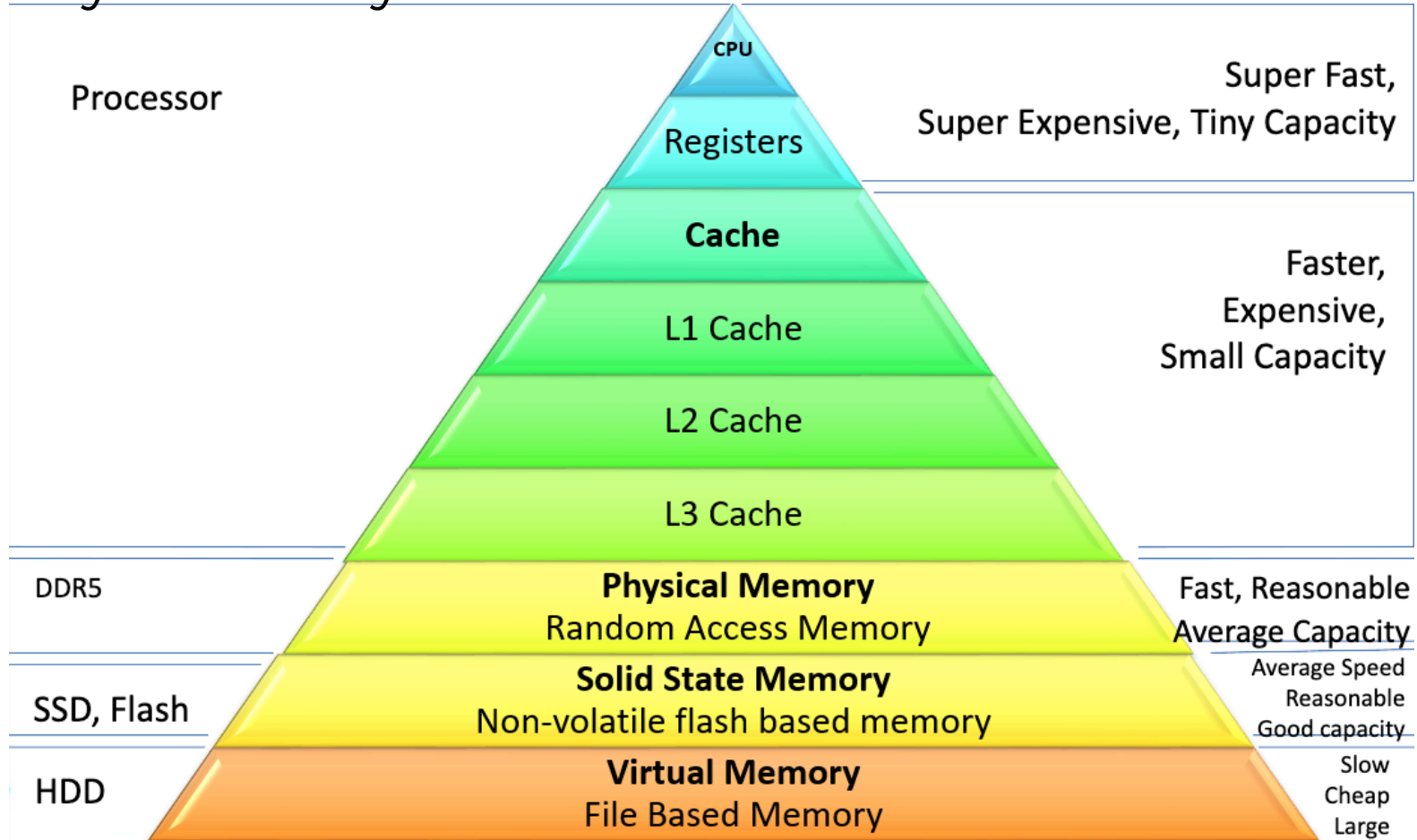
Modern Architecture



Basic Concepts

- **Process:** a task being run by a computer, often simultaneously
- **Central Processing Unit (CPU):** single or multi core chip that can handle multiple processes:
- **Control Unit (CU):** Schedules the order of instruction execution
- **Arithmetic Logic Unit (ALU):** Performs arithmetic and logical calculations
- **Floating Point Unit (FPU):** Performs operations on floating point numbers
- **Register:** small amount of storage where what is being used right now is held
- **Cache:** small amount of fast memory used to store data that is about to be used or has just been used

Memory Hierarchy



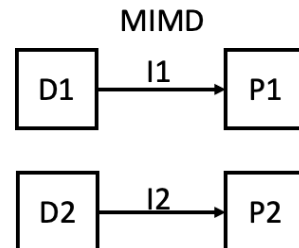
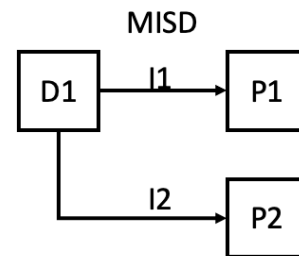
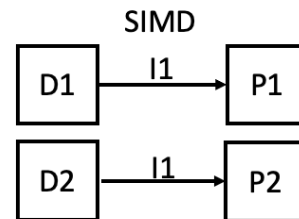
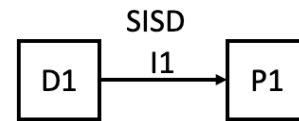
HPC Architectures

Single Instruction Single Data (SISD): a single operation is being executed and it acts on a single data item

Single Instruction Multiple Data (SIMD): multiple processors operating on its own data item, but all executing the same instruction

Multiple Instructions Single Data (MISD): multiple processors doing different operations on the same data

Multiple Instructions Multiple Data (MIMD): multiple processors doing different operations

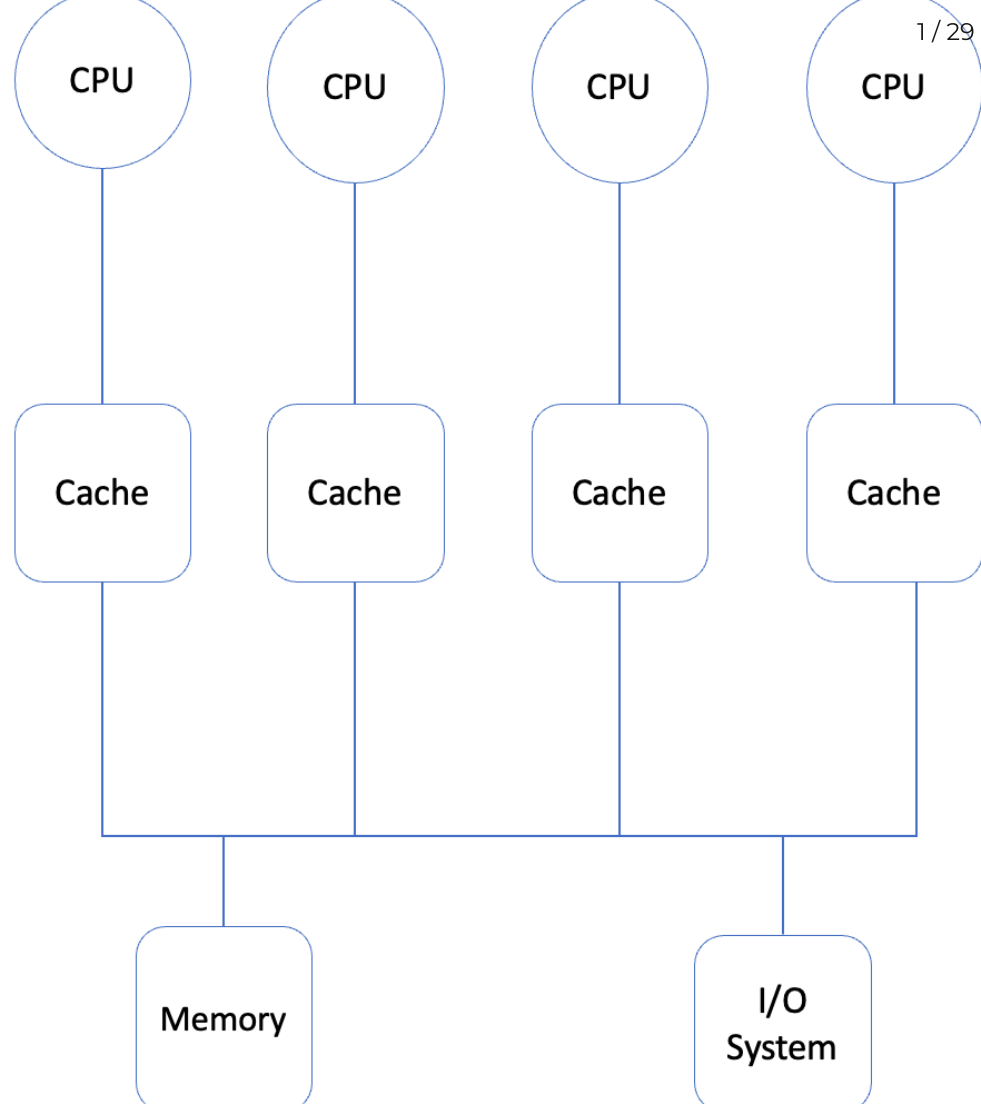


Shared Memory

Uniform Memory Access/Symmetric Multi-Processing (UMA/SMP): latency for address access is the same for each CPU

Non-Uniform Memory Access (NUMA): latency is determined by physical distance to the CPU

Cache Coherency (CC): local cache has the most up-to-date copy of a shared memory resource. Protocols must be implemented for this



Shared Memory

Advantages

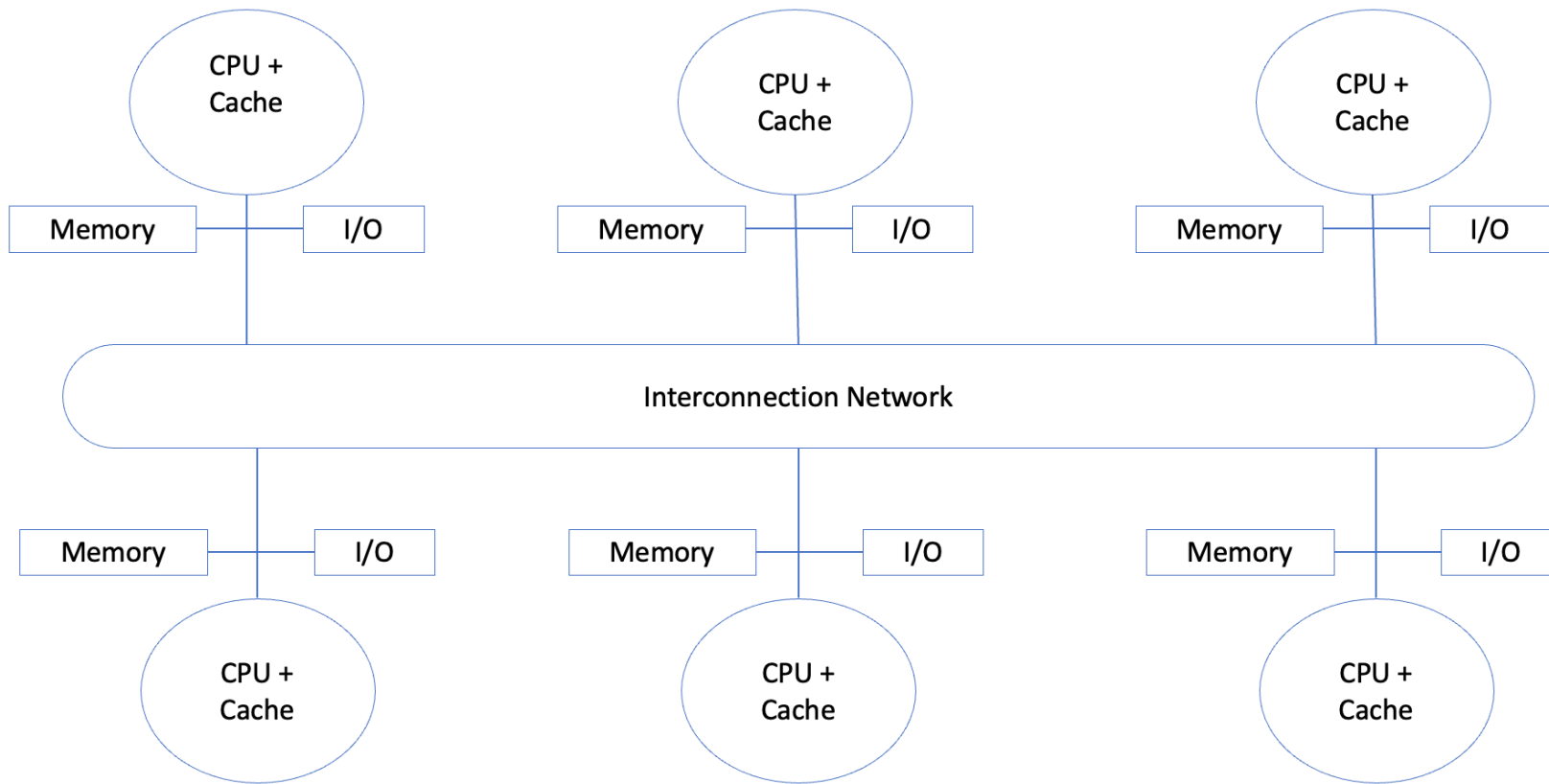
- Global shared address space provides a user-friendly programming approach to memory
- Data sharing between task is fast and uniform

Disadvantages

- Needs cache-coherency
- Lacks scalability between CPUs and memory. Adding more CPUs increases traffic on the shared memory-CPU path and increases traffic associated with cache/memory management
- Programmer must use synchronisation constructs to ensure correct access of global memory

Distributed Memory

1 / 29



Distributed Memory

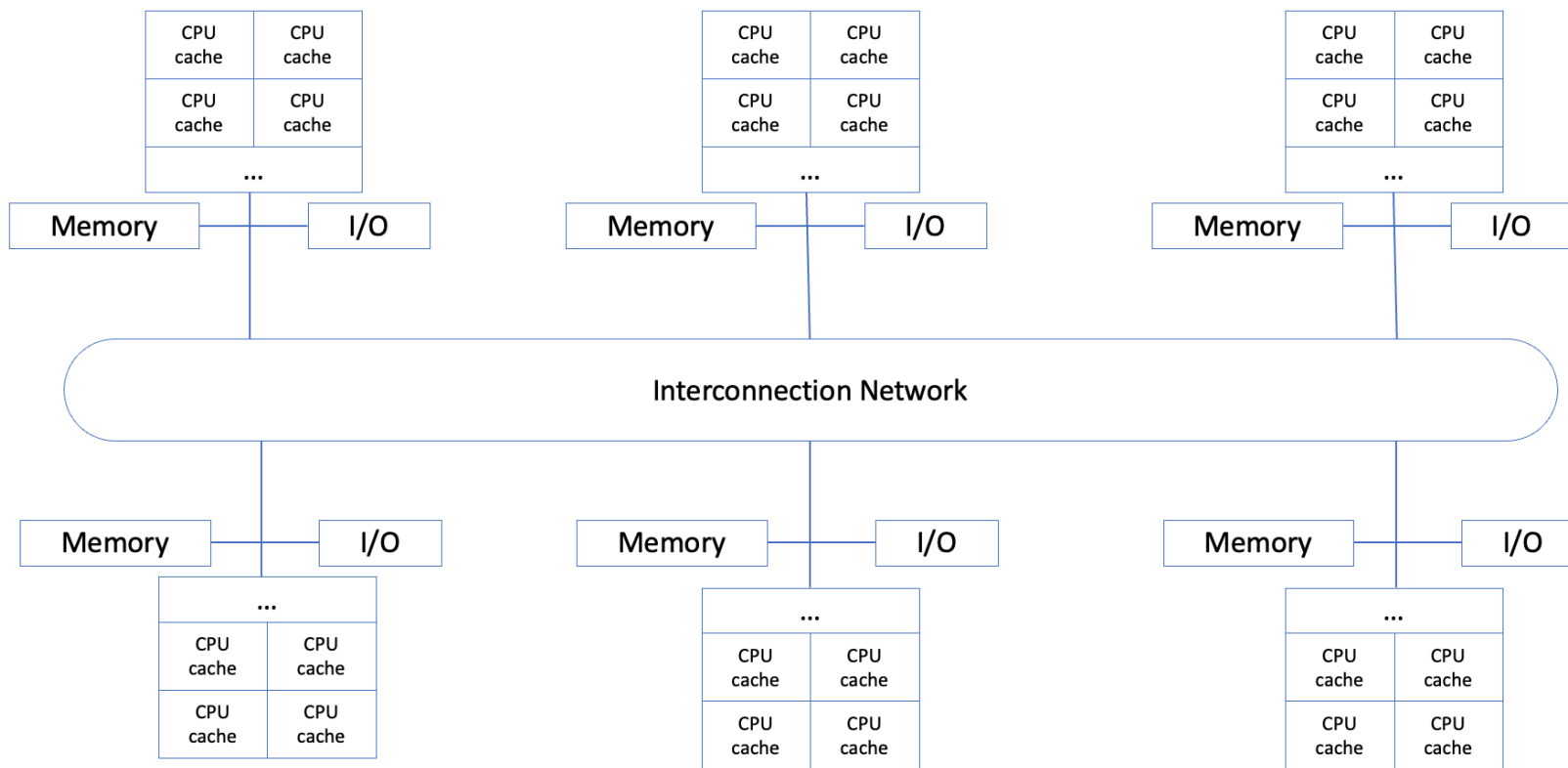
Advantages

- Memory is scalable with the number of processors
- Each processor can rapidly access its own memory without interference and overhead
- Cost-effective as consumer chips and networking can be used

Disadvantages

- Programmer must map data structures across processors
- Programmer must coordinate communication between nodes when remote data is needed
- Load balancing problems

Distributed-Shared Memory



Distributed-Shared Memory

Advantages

- Advantages of both shared and distributed memory systems
- Can use both shared and distributed memory programming paradigms

Disadvantages

- Inherits all of the disadvantages of shared and distributed memory systems

Parallel Performance

- Measured in Floating Point Operations per second (FLOPs)
- Current HPC systems operate in the petaFLOP range 10^{15} FLOPS
- The TOP500 table shows the 500 most powerful commercially available computer systems worldwide:
<https://www.top500.org/lists/top500/>
- Peak performance is given by

$$(\text{FLOPs/Cycle}) * (\text{clockrate[Hz]}) * (\text{cores}) * (\text{sockets}) * (\text{nodes})$$

- Terminology
 - **Cores** perform FLOPs.
 - The internal clock of the **core** ticks a certain number of times per second: **clock rate**
 - A **processor** contains one or more cores
 - A **socket** contains one **processor**
 - A **motherboard** contains one or more sockets
 - A **rack** contains one **motherboard**
 - A **node** contains one or more **racks**

Execution Time

- Assuming your code is correct, the goal is now to minimise the amount of time the program requires to complete
- By distributing the work over multiple CPUs we can reduce the execution time
- This requires calculations to be done in parallel and information to be exchanged between the CPUs

Speedup

- The goal of speedup is to use P processors to solve a problem P times faster. We will see later that this is usually not possible.

Relative Speedup

Compares the same parallel algorithm on one processor versus P processors and is given by

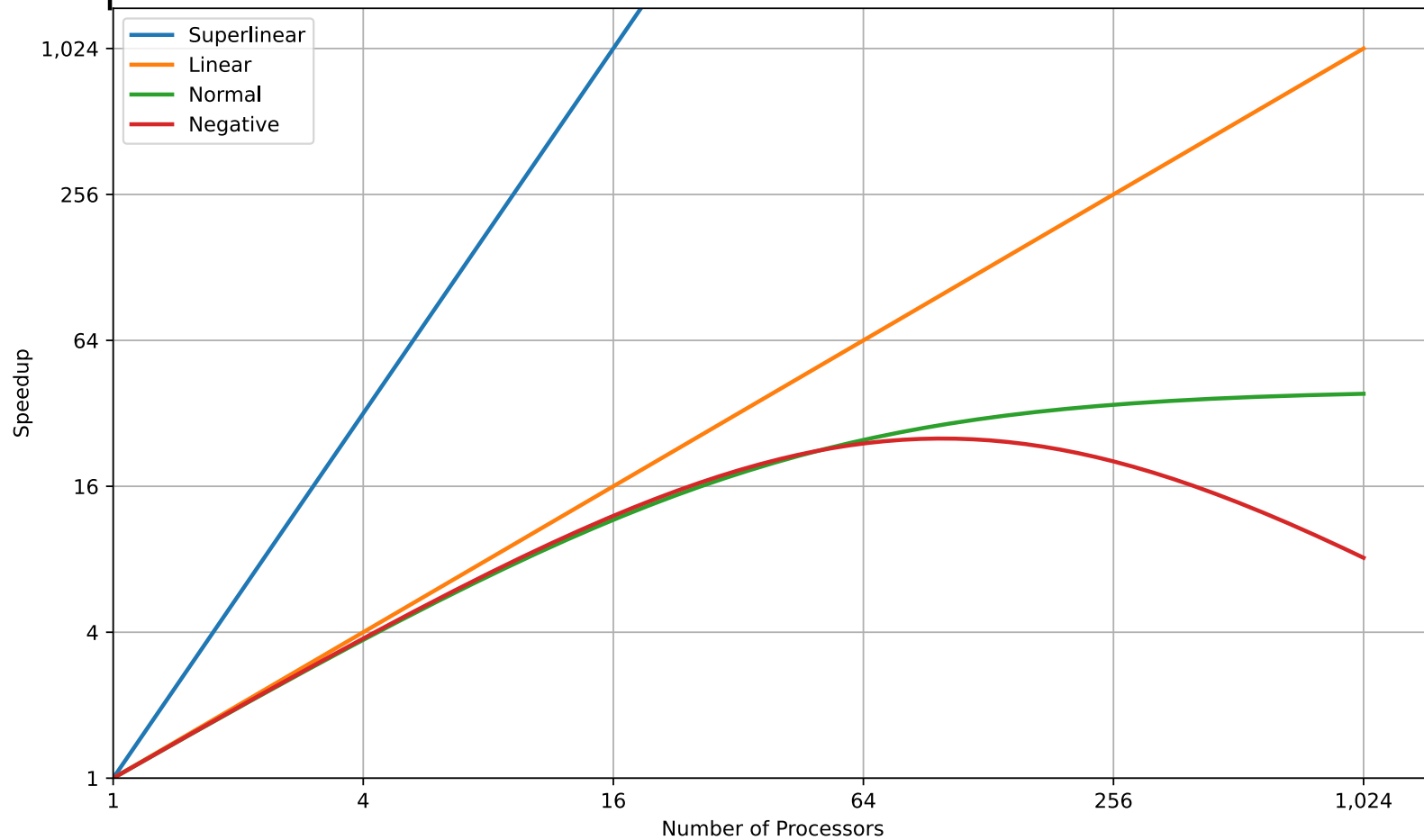
$$S_r(P) := \frac{T_{\text{par}}(1)}{T_{\text{par}}(P)}$$

Absolute Speedup

Compares the sequential algorithm on one processor versus the parallel version on P processors and is given by

$$S_a(P) := \frac{T_{\text{seq}}}{T_{\text{par}}(P)}$$

Speedup



Limits and Amdahl's Law

- Looking at the previous graph, we could ask ourselves why doubling the number of processors doesn't always half the execution time.
- This can be because of multiple things (overhead, inefficient algorithm), but the main slowdown is because of the **amount of serial/sequential operations** in the program.

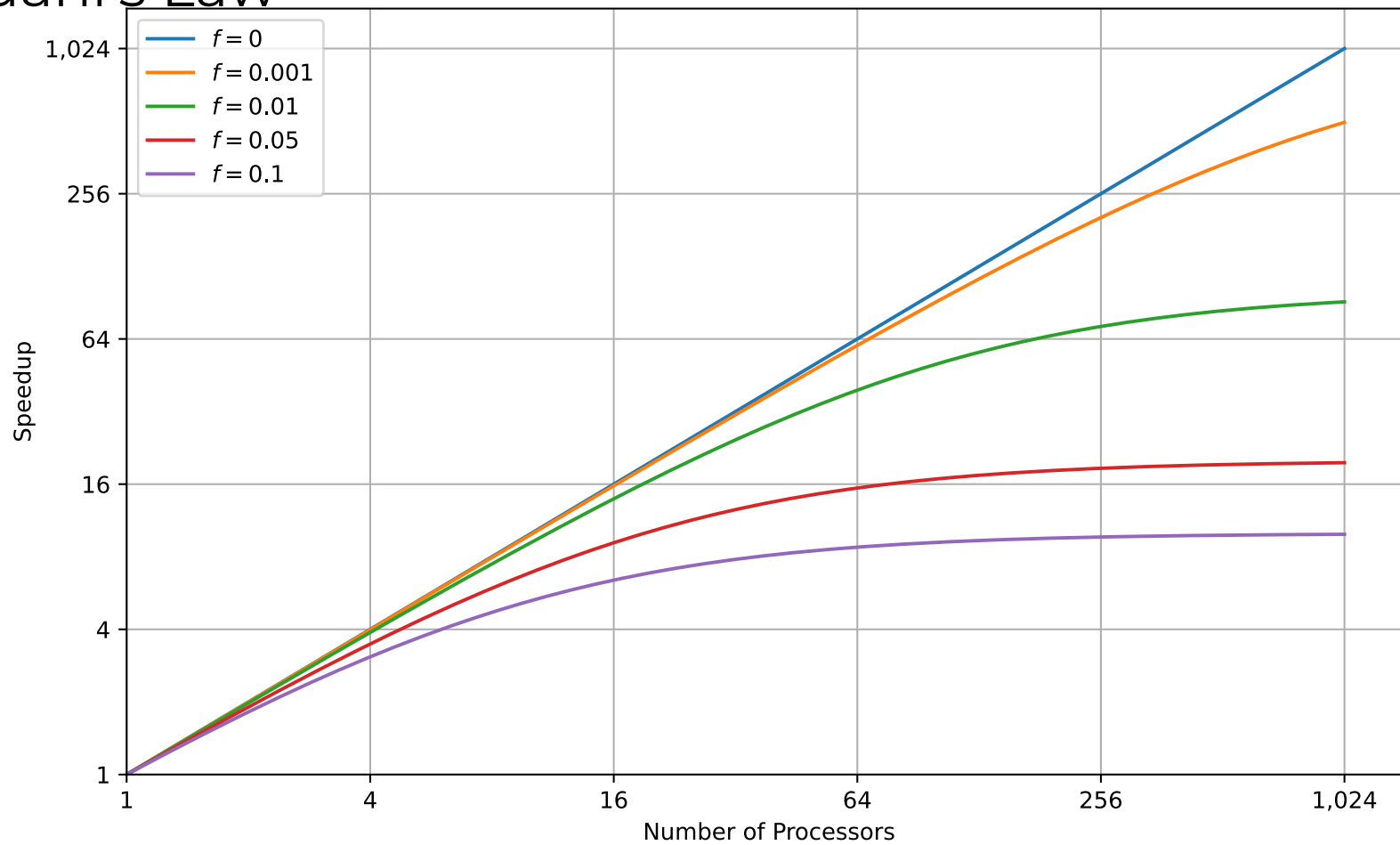
Amdahl's Law

Let f be the fraction of operations in a computation that must be performed sequentially. The maximum speedup achievable by a parallel computer with P processors is then

$$S_{\max}(f, P) = \frac{1}{f + (1 - f)/P}$$

- This assumes a **fixed** problem size

Amdahl's Law



Limits to Speedup

- Serial sections limit the parallel performance.

Parallel Overhead

Time required to coordinate parallel tasks and to communicate information between processors

- starting a thread or process
- communicating shared data
- synchronising
- extra redundant computations

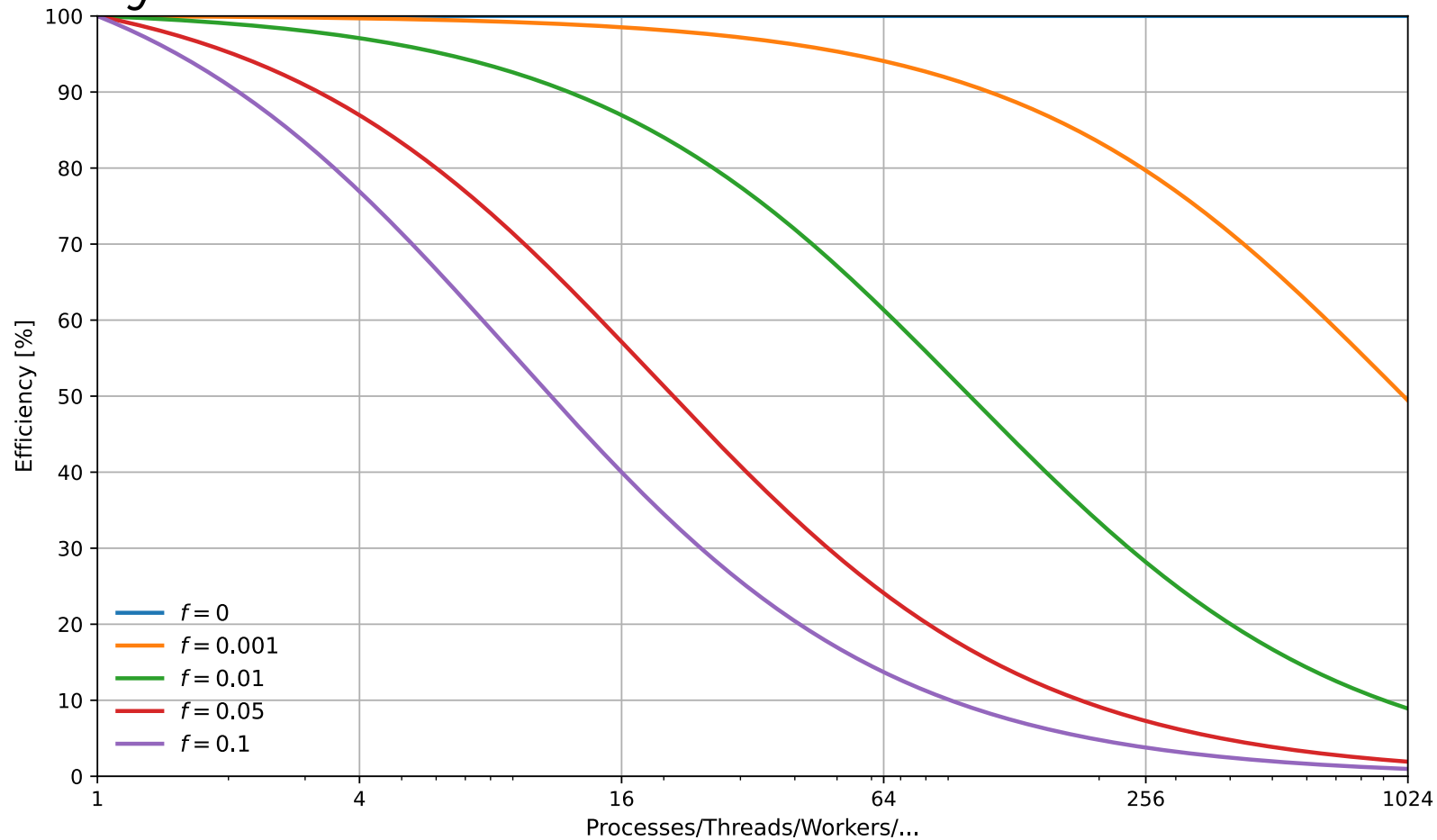
Efficiency

- A measure of how well-utilised the processors are compared to how much time is spent in communication and synchronisation

$$E_{\text{rel}}(P) = \frac{S_{\text{rel}}(P)}{P} = \frac{T_{\text{par}}(1)}{P * T_{\text{par}}(P)}$$

$$E_{\text{abs}}(P) = \frac{S_{\text{abs}}(P)}{P} = \frac{T_{\text{seq}}}{P * T_{\text{par}}(P)}$$

Efficiency



Scalability

- How does the performance change when we change the size of the problem or the number of processors? Consider a problem with N elements, that takes time T on P processors. We have two types of scaling:

Strong Scaling

Fix the problem size and increase the number of processors by a factor of k . The runtime becomes T/k . Each processor gets less work.

Weak Scaling

Increase the problem size and increase the number of processors by a factor of k . The runtime stays as T . Each processor gets the same amount of work as before.

Gustafson-Barsis' Law

- When looking at Amdahl's law, we considered a fixed problem size and looked at how the execution time varied
- Now we look at a fixed execution time and see if we can increase the problem size by increasing the number of processors

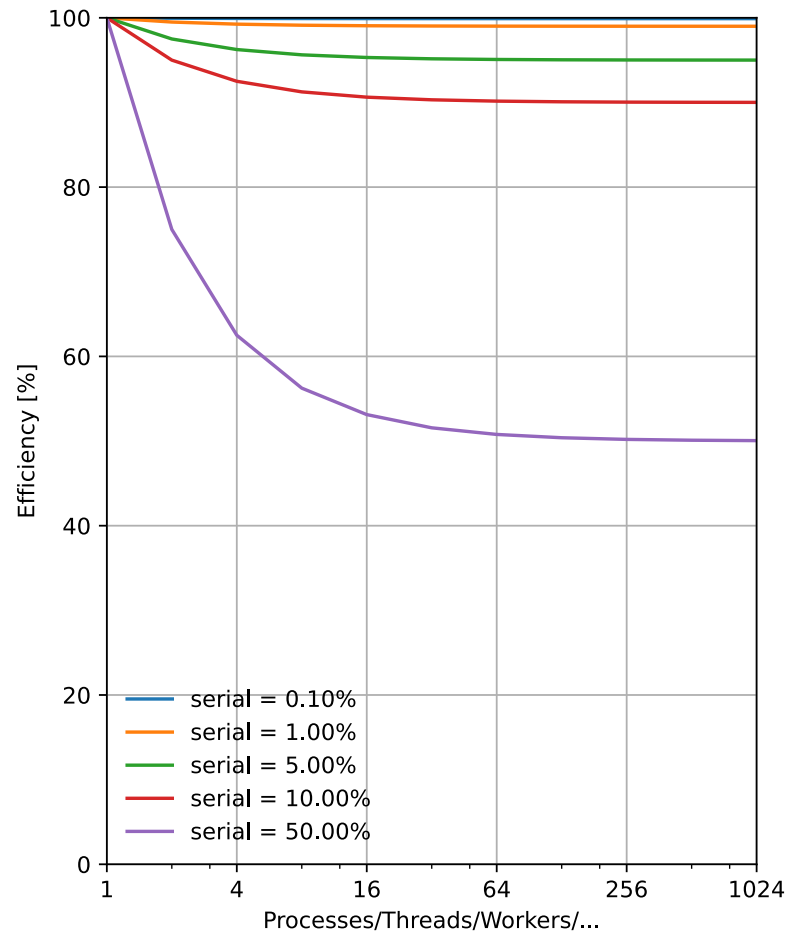
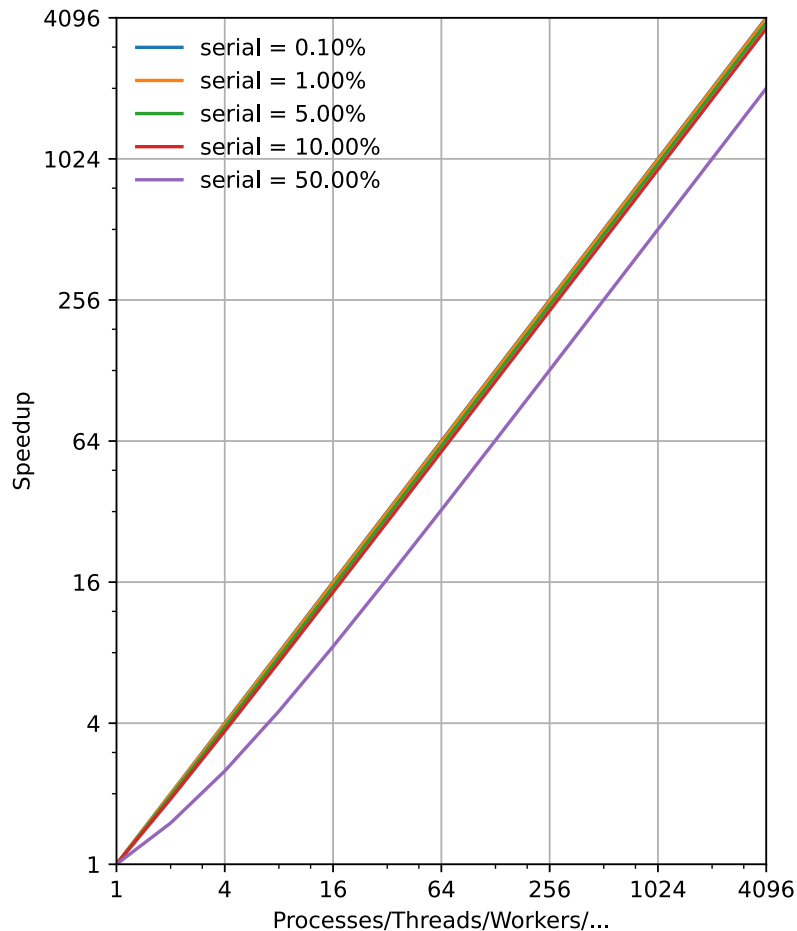
Gustafson-Barsis' Law

Given a parallel program that solves a problem of size N using P processors, let s denote the fraction of execution time spent in serial code. The maximum speedup is then given by

$$S = P + (1 - P)s$$

- The main difference between Amdahl's law and Gustafson-Barsis' law is that Amdahl's law looks at the fraction f of a sequential code that must be sequential, but Gustafson-Barsis' law looks at the fraction s of the already parallel code that must be serial. **The baseline of Amdahl is serial, but it is parallel for Gustafson-Barsis.**

Gustafson-Barsis' Law



Communication

- For some problems increasing the number of processors will:
 - decrease execution time spent computing
 - increase execution time spent communicating
- The time required for communication is dependent upon system bandwidth parameters.

The time T required to send W words between any two processors is given by

$$T = L + \frac{W}{B}$$

where

- L is latency: the fastest time in which a single byte can be sent
- B is bandwidth: how much data can be sent per unit time

Load Imbalance

- All of the laws so far assume that all processors are equally busy
- This is not always true and when processors do uneven amounts of work it is known as load imbalance

Load Imbalance Factor

$$\text{LIF} = \frac{\text{MaxLoad}}{\text{AverageLoad}}$$

- For perfectly balanced problems $\text{LIF} = 1$, but usually $\text{LIF} \geq 1$